OOP stands for Object-Oriented Programming, a paradigm based on the concept of \"objects\" which can contain data in the form attributes (also known as properties or fields) and code in the form procedures (also known as methods). It's one way to structure programs so that they are modular and reusable. Here’ endorsement:\n\nObject-Oriented Programming is a programming paradigm based on the concept of \"objects\", which can contain data, represented by fields or attributes, and code, organized into procedures called methods. In OOP languages like Java, Python, C++, etc., you define classes that encapsulate these objects along with their related properties (attributes) and behaviors (methods).\n\nKey concepts in Object-Oriented Programming include:\n\n1. **Encapsulation** - The bundling of data with the methods operating on that same data; keeping it safe from outside interference, which is a way to ensure integrity within objects. In other words encapsulation protects against changes by hiding properties and ensuring they can only be manipulated through defined functions or methods provided by their class (e.g., getters/setters).\n \n2. **Inheritance** - A mechanism where one \"child\" class inherits attributes and behaviors from a parent class, allowing for code reuse and the creation of more specialized child classes that retain common functionality while extending or altering its behavior in specific ways (e.g., through overriding methods).\n \n3. **Polymorphism** - The ability to present different interfaces with which clients can interact based on what is actually happening, allowing for flexibility and dynamic method binding at runtime when objects are cast into one another of their subclasses or implementing the same interface but behaves differently depending on its actual class type (e.g., a function that takes an object expecting any subclass instance will work with different classes).\n \n4. **Abstraction** - The concept where only necessary details about something need to be represented, hiding more complex reality behind simpler interfaces - this allows one to handle complexity by working at the interface level without concerning oneself too much about the inner complexities (e.g., defining an abstract class with some methods left unimplemented that child classes will then provide their own implementation).\n \nIn OOP, these principles enable you to create more organized and modular code where components can be easily reused across different parts of a software system or even among multiple projects (as long as they share the same language with compatible interfaces), improving maintainability. They also allow programmers to model real-world entities effectively which makes it easier for non-programmers, like designers and managers, to understand how applications are structured internally since OOP often mirrors our understanding of physical objects interacting in a tangible way (e.g., cars have methods related to driving, parking etc.).